Engineering Memory System

Engineering knowledge should compound, not disappear.

1. Executive Summary

Software engineering is undergoing a fundamental shift. Large language models have rapidly evolved from code-completion tools into capable software engineering assistants that can navigate repositories, modify large codebases, execute tools, and perform increasingly sophisticated implementation tasks. As these capabilities continue to improve, code generation itself is becoming progressively less scarce. The limiting factor is no longer the ability to produce code, but the ability to accumulate engineering understanding over time.

Human engineers derive much of their effectiveness from persistent knowledge rather than raw reasoning ability. Every debugging session, architectural investigation, design review, or implementation attempt permanently changes how an experienced engineer understands a system. Over time, this accumulated understanding becomes an internal model of the codebase that guides future engineering decisions. This model includes architectural constraints, implementation trade-offs, historical context, operational experience, debugging intuition, and knowledge of why previous decisions were made. It compounds continuously throughout the lifetime of a project.

Contemporary AI coding agents possess very little of this persistence. Although they can reason effectively within the lifetime of a single interaction, most of the engineering understanding developed during that interaction disappears once the session ends. Future sessions frequently reconstruct context that has already been established, rediscover implementation decisions that were previously made, and repeat investigations whose conclusions are already known to the engineer. **The resulting workflow is fundamentally stateless despite the inherently stateful nature of software engineering.**

Engineering Memory System (EMS) is an attempt to address this limitation. Rather than treating memory as conversation history or prompt compression, EMS treats memory as accumulated engineering cognition. The system continuously captures engineering understanding generated during software development and organizes it into three complementary forms of knowledge: **historical memory** describing how a repository evolved over time, **canonical memory** representing reviewed engineering knowledge that should persist indefinitely, and **session memory** containing temporary understanding accumulated during active implementation work.

This distinction is central to the design. Conversations are transient; engineering conclusions are durable. The objective of EMS is therefore not to preserve interactions, but to preserve the understanding that emerges from those interactions. Future coding sessions inherit this

accumulated knowledge, allowing engineering understanding to compound instead of being reconstructed from scratch.

The long-term motivation behind EMS extends beyond improving individual coding sessions. As software engineering becomes increasingly AI-native, persistent engineering cognition may become a foundational layer of the development stack, analogous to the role version control systems play today. Models will continue to improve, context windows will continue to grow, and code generation will continue to become more reliable. What will remain scarce is accumulated engineering understanding. EMS is built around the assumption that this understanding, rather than code generation itself, will become the defining competitive advantage of future AI software engineering systems.

Recent coding agents have made code generation abundant. As that bottleneck disappears, engineering continuity becomes increasingly important. The value of persistent engineering cognition grows alongside the capability of the agents themselves, making this the right time for an independent engineering memory layer.

2. The Observation

The idea behind EMS did not originate from a large architectural insight or a research problem. It emerged from a relatively ordinary interruption during day-to-day development.

While working on a feature using OpenCode as my primary development environment, my terminal encountered a permissions issue that abruptly terminated the session. Because the work was already several prompts into the implementation, the only practical way to continue was to open a new terminal and manually copy every prompt and response into the new session so that the agent could reconstruct enough context to proceed. A short time later, the same failure occurred again, requiring the entire process to be repeated.

Initially, this appeared to be little more than an inconvenience. After repeating the process several times, however, a more fundamental limitation became apparent. The coding agent possessed no persistent understanding of the work it had already performed. Every session existed in isolation, and transferring engineering knowledge between sessions required replaying the complete interaction history. Once the conversation disappeared, so did the engineering understanding that had been developed throughout it.

The most straightforward solution seemed obvious: preserve everything. Conversations could be stored, code diffs archived, and future sessions allowed to retrieve previous interactions whenever necessary. Many existing systems already pursue variations of this approach. The more I considered it, however, the less convincing it became. Conversations grow without bound, code diffs quickly become expensive to search and reason about, and neither representation captures the property that actually makes experienced engineers more effective. They preserve what happened rather than what was learned.

An experienced engineer rarely recalls every terminal command executed while debugging a production incident. Instead, they retain the underlying engineering conclusion. They remember that a particular abstraction fails under specific conditions, that a subsystem exhibits a non-obvious constraint, or that an architectural decision exists because several alternatives had already been explored and rejected. Those conclusions influence future engineering decisions long after the original implementation details have faded from memory.

This observation fundamentally changed the direction of the project. Rather than attempting to preserve software development as a sequence of conversations, EMS would attempt to preserve the engineering understanding produced by those conversations. The objective shifted from storing interactions to accumulating cognition.

As the design evolved, another observation naturally followed. Engineering knowledge does not exist at a single temporal scale. Some knowledge is useful only while solving the current problem. Other knowledge becomes a permanent characteristic of the repository. Still other knowledge exists because of the historical evolution of the system over hundreds or thousands of commits. Treating all of these forms of knowledge identically would inevitably lead either to excessive accumulation of temporary information or to the loss of valuable long-term understanding.

This ultimately led to the three-layer memory model that forms the basis of EMS. Session memory captures understanding generated while solving an active engineering task. Canonical memory stores reviewed engineering knowledge that should influence future development regardless of session boundaries. Historical memory records how the repository evolved and why particular decisions were made over time. Together, these layers attempt to model engineering knowledge in a manner that more closely resembles the way experienced software engineers accumulate and organize understanding throughout the lifetime of a project.

The central objective of EMS therefore became significantly narrower and more precise than persistent chat history. It is an attempt to ensure that engineering understanding, once acquired, becomes part of the permanent cognitive state available to every future AI-assisted development session.

3. Engineering Cognition as an Infrastructure Layer

Software engineering has repeatedly progressed by moving important abstractions into infrastructure: version control externalized history, CI/CD externalized deployment, cloud computing externalized hardware, and AI coding agents are beginning to externalize implementation. Engineering understanding remains one of the few capabilities that still exists

almost entirely inside individual engineers. EMS is built on the hypothesis that engineering cognition itself should become infrastructure.

More recently, AI coding agents have begun externalizing software implementation itself. Engineers increasingly specify intent while models perform substantial portions of code generation, refactoring, navigation, and testing. As model capabilities continue to improve, the human role gradually shifts from producing code toward supervising, reviewing, and directing increasingly autonomous engineering systems.

One aspect of software engineering, however, remains almost entirely internal to the engineer: accumulated understanding. Architectural intuition, debugging experience, implementation trade-offs, operational knowledge, and the historical reasoning behind design decisions continue to reside primarily in the minds of the engineers who produced them. Although fragments of this understanding appear in documentation, commit messages, design documents, and conversations, the cognitive model itself is rarely captured in a structured form that can be continuously reused.

This limitation becomes increasingly significant as AI systems assume larger roles in software development. The effectiveness of an experienced engineer is determined not solely by reasoning ability, but by the body of engineering understanding accumulated through repeated interaction with a system. Today's AI coding agents largely lack this accumulated state. Each session begins with limited awareness of previous investigations, architectural discoveries, or implementation history, despite the fact that these conclusions already exist within the engineering organization.

EMS is built on the hypothesis that engineering cognition should itself become infrastructure. Rather than existing exclusively within individual engineers or transient conversations, engineering understanding can be continuously accumulated, reviewed, organized, and made available to future AI-assisted development sessions. Under this view, persistent engineering memory is not an auxiliary feature of coding agents, but a foundational capability that enables engineering knowledge to compound over the lifetime of a software system.

4. Design Requirements for Persistent Engineering Cognition

If the objective is to build a system capable of accumulating engineering understanding over time, it is useful to first define what such a system must satisfy. These requirements arise naturally from observing how experienced software engineers acquire, retain, and apply knowledge while working on long-lived codebases.

The first requirement is that engineering knowledge must be represented as understanding rather than interaction history. Conversations, prompts, and code edits are

valuable because they produce conclusions, not because they are valuable artifacts in themselves. A persistent cognition system should therefore capture the engineering knowledge that emerges from development rather than the sequence of events that produced it.

Second, engineering knowledge exists at multiple temporal scales. Some knowledge is relevant only while solving the current implementation task, while other knowledge should persist throughout the lifetime of the repository. Historical information describing how a system evolved serves yet another purpose, providing context that explains why the current architecture exists. Treating all knowledge identically inevitably produces either excessive accumulation of transient information or insufficient preservation of long-term engineering understanding. Any practical system therefore requires multiple forms of memory with distinct lifecycles.

Third, accumulated knowledge must remain reviewable. Human engineers continuously refine their understanding of a system, discarding incorrect assumptions while reinforcing accurate ones. A persistent cognition layer cannot assume that every inference generated during development is correct. Instead, it should support an explicit mechanism through which engineering knowledge can be promoted, refined, corrected, or rejected over time. Persistent knowledge should emerge through continuous review rather than automatic accumulation.

Fourth, retrieval should prioritize relevance over completeness. As engineering knowledge compounds over months or years, loading the entirety of accumulated knowledge into every interaction becomes computationally infeasible while simultaneously reducing model performance. The system must instead identify which portions of accumulated engineering understanding are most relevant to the current task and retrieve only those. The objective is not to maximize retrieved information, but to maximize useful engineering context.

Fifth, engineering cognition should be independent of any particular language model or coding environment. Foundation models will continue evolving, context windows will increase, and new agent frameworks will emerge. Engineering understanding, however, belongs to the engineer rather than to the model being used at any particular moment. A cognition layer should therefore exist independently of the surrounding AI ecosystem and remain portable across models, coding agents, and development environments.

Finally, accumulated engineering knowledge should improve continuously with use. Every debugging investigation, implementation task, architectural review, and production incident should increase the quality of future engineering decisions. A persistent cognition system is therefore fundamentally compounding. The longer it participates in software development, the more complete and useful its representation of the system becomes.

These requirements collectively define the problem EMS attempts to solve. Rather than designing another storage mechanism for conversations, the objective becomes constructing a persistent engineering knowledge system whose behavior more closely resembles the continuously evolving mental model maintained by experienced software engineers.

5. Why Existing Solutions Don't Solve It

Nearly every modern AI agent has begun introducing some notion of persistent memory. Although these systems differ substantially in implementation, they largely share one underlying assumption: memory is information that should be extracted, stored, and later retrieved. This assumption has produced meaningful improvements over completely stateless interactions, but it does not fully address the problem of persistent engineering cognition.

The first category consists of **conversation persistence systems**, which preserve previous interactions so they can be replayed or revisited later. Rather than losing an entire session when it ends, the agent can retrieve earlier conversations and reconstruct prior reasoning. This significantly improves continuity compared to completely stateless agents, but conversations themselves are not engineering understanding. They contain exploratory reasoning, failed ideas, intermediate questions, implementation details, and temporary hypotheses whose primary value was producing an engineering conclusion. Replaying the conversation often requires the model to reconstruct understanding that had already existed.

A second category focuses on **memory extraction**. Systems such as Mem0 continuously identify important facts from conversations, consolidate them, and retrieve them when future tasks appear semantically related. More recent systems extend this idea using graph-based representations to capture richer relationships between stored facts. These approaches substantially improve personalization, long-term recall, and token efficiency by remembering user preferences, historical interactions, and previously established information.

A third category extends memory into **persistent agent state**. Systems such as Hermes Agent maintain curated memory files, searchable session history, user profiles, and integrations with external memory providers such as Mem0, Graphiti, Cognee, and others. These systems allow agents to remember project conventions, user preferences, environment configuration, and previous conversations across sessions, making the agent itself substantially more persistent over time.

These systems are valuable. They solve the problem they were designed to solve.

However, they all optimize the preservation and retrieval of **information**.

EMS asks a different question.

Experienced software engineers rarely solve new problems by replaying previous conversations or searching for isolated facts. Instead, they rely on an internal model that has been continuously refined through implementation experience. This model contains architectural intuition, debugging experience, implementation trade-offs, historical context, and—most

importantly—an understanding of **why** previous engineering decisions were made. Individual conversations contribute to this understanding, but they are not the understanding itself.

The central premise behind EMS is therefore that **engineering cognition should be treated as a first-class computational representation rather than an emergent property of archived interactions**. Instead of preserving conversations indefinitely or retrieving isolated facts, EMS continuously attempts to distill engineering understanding from development activity, organizes that understanding according to its role within the software lifecycle, and makes it available to future engineering sessions through relevance-based retrieval.

This distinction is subtle but fundamental.

Existing systems primarily optimize **memory**.

EMS attempts to optimize **understanding**.

Memory answers: *What happened before?*

Engineering cognition attempts to answer: *What has already been learned that should influence the engineering decision being made now?*

EMS is therefore not simply another memory system. It is an attempt to investigate whether accumulated engineering understanding can itself become a reusable computational object whose value compounds across engineering sessions.

6. EMS Architecture

Here's the entire EMS architecture in a diagram:

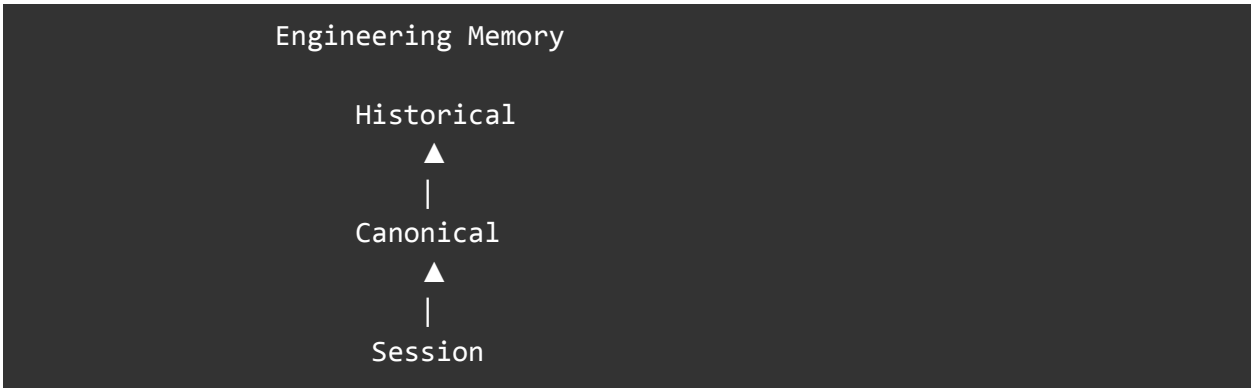
```
Repository
↓
Historical Builder
↓
Historical Memory
↓
Engineering Session
↓
Session Knowledge
↓
Knowledge Extraction
↓
Candidate Proposals
```

```
↓
Developer Review
↓
Canonical Memory
↓
Retrieval + Ranking
↓
Prompt Augmentation
↓
Coding Agent
```

6.1 Memory Hierarchy

Engineering knowledge evolves across multiple temporal scales. Temporary implementation observations gradually become durable engineering understanding, while long-term repository evolution provides historical context that influences future reasoning. Representing all of these forms of knowledge within a single memory store inevitably mixes transient observations with permanent engineering conclusions.

EMS instead organizes engineering cognition into a hierarchical memory model.



Session Memory captures knowledge generated while solving an active engineering problem, including debugging discoveries, implementation decisions, architectural observations, and failed approaches. It represents the working cognition of the current development session.

Canonical Memory stores reviewed engineering knowledge that has been accepted as durable understanding of the codebase. This layer forms the persistent engineering model inherited by future sessions and grows continuously as engineers interact with the repository.

Historical Memory represents knowledge extracted from the evolution of the repository itself. By analyzing commit history and implementation lineage, EMS reconstructs the architectural

context that explains how the current system came to exist, providing future sessions with engineering history that would otherwise exist only in the minds of long-term contributors.

Together, these layers separate transient reasoning from durable engineering understanding while allowing knowledge to mature naturally over the lifetime of a project.

6.2 Knowledge Lifecycle

Engineering cognition should not be treated as static documentation. It is a continuously evolving representation of engineering understanding that matures through repeated software development activities. Every implementation task, debugging investigation, or architectural exploration contributes new observations that may eventually become permanent engineering knowledge.

EMS models this process as an explicit knowledge lifecycle.

```
Engineering Session

↓

Engineering observations

↓

Knowledge extraction

↓

Candidate proposals

↓

Developer review

↓

Canonical memory

↓
```

```
Context Retrieval

↓

Next engineering session
```

Each engineering session produces observations that are distilled into candidate knowledge objects rather than preserving the underlying conversations. These proposals capture engineering conclusions together with supporting evidence, provenance, semantic metadata, and confidence estimates. Before becoming part of long-term engineering cognition, every proposal is reviewed by the developer, ensuring that canonical memory reflects validated engineering understanding rather than unverified model inferences.

The resulting canonical knowledge becomes available during context construction for subsequent engineering sessions, allowing each completed task to improve the quality of future reasoning. In this manner, engineering understanding compounds continuously throughout the lifetime of the repository instead of being repeatedly reconstructed from historical interactions.

```
Engineer prompt
"The authentication service occasionally enters a retry loop after token
expiration."

↓

LLM response
"The retry loop occurs because TokenManager refreshes JWTs before
invalidating the cache."
↓

EMS proposal

Authentication retries occur before cache invalidation.
Evidence:
- token_manager.rs
- auth.rs
Confidence: 0.91

↓

Developer accepts
```

```
↓

Canonical memory
Authentication flow assumes cache invalidation happens
after successful JWT refresh.
```

6.3 Context Construction

A persistent engineering knowledge base is only useful if the relevant knowledge can be retrieved efficiently at inference time. As the memory graph grows, naively injecting all accumulated knowledge into every prompt rapidly exceeds available context windows while simultaneously degrading reasoning quality.

EMS therefore constructs context dynamically for every engineering request. Rather than retrieving documents based solely on semantic similarity, the retrieval pipeline attempts to identify the subset of engineering knowledge most likely to improve reasoning for the current task.

```
Engineer Prompt
  │
  ▼
Intent Analysis
  │
  ▼
Candidate Retrieval
  │
  ▼
Ranking
  │
  ▼
Prompt Augmentation
  │
  ▼
Coding Agent
```

The prompt is first analyzed to infer its engineering intent. Candidate knowledge is then retrieved across the different memory layers using semantic indexing and metadata. Retrieved documents are ranked according to their relevance, provenance, confidence, and engineering

importance before being assembled into a bounded context that respects configurable token budgets.

This architecture ensures that each interaction receives the most relevant engineering understanding rather than the largest possible amount of historical information. As the memory system grows, retrieval complexity increases while prompt size remains bounded.

6.4 Human-in-the-loop Cognition

A fundamental design decision within EMS is that permanent engineering knowledge should never be accepted automatically.

Language models are capable of producing plausible engineering conclusions, but plausibility alone is insufficient for long-lived cognition. If every inferred fact were allowed to enter persistent memory without review, incorrect assumptions would accumulate alongside correct ones, gradually reducing the quality of the knowledge base.

EMS therefore introduces an explicit review stage between temporary reasoning and permanent cognition.

Every engineering session generates candidate knowledge proposals extracted from the work performed during that session. These proposals remain provisional until reviewed by the engineer responsible for the work. Only accepted proposals become part of canonical memory, while rejected proposals remain isolated from future retrieval.

This review process serves two purposes. First, it preserves the accuracy of long-term engineering knowledge by preventing hallucinated or speculative conclusions from becoming permanent. Second, it allows canonical memory to evolve according to the collective engineering understanding of the team rather than the transient reasoning of an individual model.

In this sense, EMS does not attempt to automate engineering judgment. It attempts to augment it. Models generate candidate cognition; engineers determine what becomes organizational knowledge.

6.5 Why the Architecture is Agent-Agnostic

EMS is intentionally designed as infrastructure rather than as a feature of any individual coding agent.

The system stores engineering cognition independently of the language model, development environment, or orchestration framework responsible for producing it. Today, EMS integrates only with OpenCode, but future integrations would target Claude Code, Codex, Cursor, etc. EMS shall continue to support entirely different systems as the AI tooling ecosystem evolves.

This separation is deliberate. Foundation models will improve, coding agents will change, and development workflows will continue to evolve. Engineering understanding, however, belongs to the engineer and the codebase rather than to the interface through which that understanding was produced.

By separating cognition from execution, EMS allows accumulated engineering knowledge to survive changes in models, tooling, and development environments. The engineer's understanding compounds continuously even as the surrounding AI ecosystem changes.

The long-term objective is therefore not to build a better coding agent. It is to provide a persistent engineering cognition layer that any coding agent can inherit, allowing engineering knowledge to outlive the individual systems that generated it.

7. Limitations

EMS remains an evolving research system and several open problems remain.

First, extracting engineering understanding rather than implementation facts is itself an unsolved problem. Current language models frequently identify low-level observations that are syntactically correct but do not meaningfully improve future engineering reasoning.

Second, retrieval quality is fundamentally constrained by indexing quality. Even a well-structured memory hierarchy provides little value if relevant knowledge cannot be surfaced consistently for the current task.

Third, context construction remains bounded by the available context window of the underlying model. Although EMS prioritizes knowledge using relevance ranking and configurable token budgets, important engineering knowledge may occasionally be omitted from a given interaction.

8. Conclusion

Software engineering is increasingly becoming a collaborative process between humans and language models. As model capabilities continue to improve, the scarcity of code generation is steadily diminishing. What remains scarce is accumulated engineering understanding.

Human engineers become more effective over time because every implementation, debugging session, architectural investigation, and production incident permanently refines their internal model of the software system. That continuously evolving understanding—not the ability to write syntax—is what allows experienced engineers to solve increasingly complex problems.

Current AI coding agents largely lack this property. Their reasoning is powerful, but their engineering cognition is transient.

Engineering Memory System is an attempt to externalize that cognition into persistent infrastructure. Rather than preserving conversations, EMS attempts to preserve engineering understanding itself, allowing knowledge to accumulate across repositories, branches, sessions, and years of software evolution while remaining independent of any individual language model or coding agent.

Whether persistent engineering cognition becomes a standard abstraction for AI-native software engineering remains to be seen. EMS is one attempt to explore that hypothesis through practical system design.

Version control made software collaborative.

Persistent engineering memory may make software cumulative.

Author: Mudit Sarda
Date: 29 June, 2026